# SIMATS ENGINEERING
## ASSESSMENT TOOL 1

**COURSE CODE: CSA1442** **COURSE NAME: Compiler Design for Higher Level Processing**

---

**COURSE OUTCOME COVERED**

***CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation.*** *(BL3)*

*Assessment Tool Weightage: 40%*

---

**QUESTIONS:4** **Total Marks:50**

Annexure A

## DESIGN TASK

---

***Code of Conduct:***

*I ____Juhi Surin (192525148)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
*I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student:*** *Juhi Surin*

***Course Faculty***

Annexure A

# DESIGN TASK

## Q1. Predictive Parser Design (Application Level)

**Task:**
*Construct an LL(1) predictive parsing table for the given grammar and demonstrate its application by parsing the specified input string step-by-step, clearly showing stack operations, input symbols, and parsing actions.*
.
**Grammar:**
E → T E'
E' → + T E' | ε
T → F T'
T' → * F T' | ε
F → (E) | id
**Conditions:**

- Remove left recursion (if any)
- Compute FIRST and FOLLOW sets
- Construct LL(1) parsing table

**Test Case:**
**Input String:**
id + id * id
**Expected Output:**

- Step-by-step parsing actions
- Final result: **String Accepted**

**Q2. Shift-Reduce Parsing (Application Level)**
**Task:**
*Apply shift-reduce parsing to the given grammar and input string, and demonstrate the complete parsing process by showing each step with stack contents, remaining input, and parsing actions (shift, reduce, accept, or error), clearly indicating all reductions and decisions made during parsing.*

**Grammar:**
E → E + E
E → E * E
E → id
**Conditions:**

- Show stack, input buffer, and action at each step
- Resolve conflicts using precedence (* has higher precedence than +*)

**Test Case:**
**Input:**
id + id * id
**Expected Output:**

- Sequence of actions: Shift / Reduce / Accept
- Final result: **String Accepted**

Integrated Design Question (SLR, LALR, CLR Parsing)

***Q3.Construct the LR parsing tables for the given grammar and perform a detailed comparison of different LR parsing techniques (SLR, LALR, and CLR), highlighting their construction steps, state transitions, table sizes, and any conflicts encountered.***

S → L = R | R
L → * R | id
R → L

**Tasks**

**(i) CLR (Canonical LR) Design**

Construct the canonical collection of LR(1) items
Build the CLR parsing table
Show all item sets and transitions

**(ii) SLR Parser Design**

Compute FIRST and FOLLOW sets
Construct the SLR parsing table
Identify and explain any shift-reduce or reduce-reduce conflicts

**(iii) LALR Parser Design**

Merge compatible LR(1) states
Construct the LALR parsing table
Compare with CLR and show how states are reduced

**(iv) Parsing Demonstration**

Using any one of the constructed parsing tables,
parse the input string:
id = * id
Show:
Stack
Input buffer
Action (Shift / Reduce / Accept)

Q4. Critically evaluate the effectiveness of Parsing Expression Grammars (PEG) as a modern parsing technique by comparing them with traditional LL and LR parsers in terms of ambiguity handling, grammar expressiveness, implementation complexity, performance, and real-world applicability.

Your answer should:

1. **Analyze** how PEG parsers resolve ambiguity differently from CFG-based parsers
2. **Compare** PEG with LL and LR parsing in terms of:
    - Grammar expressiveness
    - Ease of implementation
    - Performance and backtracking
3. **Examine** real-world adoption (e.g., Python's PEG parser) and justify why PEG was preferred
4. **Critically evaluate** limitations of PEG (e.g., lack of left recursion support, ordered choice behavior)
5. **Conclude** whether PEG is suitable for all programming languages or only specific cases, with justification

## Q 5. Advanced LL(1) Parser Evaluation and Design

### Task:

*Analyze and redesign the given grammar to make it suitable for LL(1) parsing, construct the predictive parsing table, and evaluate its correctness by parsing the input string.*

### Grammar:

S → S + T | T
T → T * F | F
F → (S) | id

### Conditions:

- Eliminate left recursion completely
- Apply left factoring where necessary

- Compute FIRST and FOLLOW sets accurately
- Construct the LL(1) parsing table
- Justify whether the grammar satisfies LL(1) conditions
- Trace full parsing steps with stack, input, and action

**Test Case:**
Input: id + id * id
Expected: String Accepted with full trace

---

**Q6. Comparative Analysis of SLR vs LALR vs CLR**

**Task:**
*Construct and compare SLR, LALR, and CLR parsing tables for the given grammar and evaluate their differences in handling conflicts and efficiency.*

**Grammar:**
S → L = R | R
L → * R | id
R → L

**Conditions:**

- Construct LR(0), LR(1) item sets
- Compute FOLLOW sets (for SLR)
- Build SLR, LALR, and CLR tables
- Identify and explain conflicts in each method
- Compare number of states and table sizes
- Justify which method is most efficient

**Test Case:**
Input: id = * id
Expected: Valid parse with comparison analysis

---

**Q7. Operator Precedence Parsing Design and Evaluation**

**Task:**
*Design an operator precedence parser and evaluate its correctness for handling precedence and associativity.*

**Grammar:**
E → E + E | E * E | (E) | id

**Conditions:**

- Define operator precedence relations (<, =, >)
- Construct precedence table
- Handle associativity rules explicitly
- Detect and resolve conflicts
- Show step-by-step parsing actions
- Evaluate correctness of parsing decisions

**Test Case:**
Input: id + id * id

---

**Q8. Ambiguity Removal and Parsing Strategy Evaluation**

**Task:**
*Analyze the ambiguity in the given grammar, redesign it to remove ambiguity, and evaluate parsing using shift-reduce parsing.*

**Grammar:**
E → E - E | E / E | id

**Conditions:**

- Identify ambiguity in the grammar
- Redesign grammar using precedence rules
- Justify associativity decisions
- Apply shift-reduce parsing
- Show all stack operations and reductions
- Evaluate correctness of final parse

**Test Case:**
Input: id - id / id

---

**Q9. PEG vs CFG Parsing Strategy Evaluation**

**Task:**
*Design a PEG-based parser for the given expression grammar and critically evaluate its behavior compared to CFG-based parsing.*

**Grammar (CFG form):**
E → E + T | T
T → T * F | F
F → id

**Conditions:**

- Convert CFG into PEG form
- Explain ordered choice behavior
- Demonstrate parsing steps
- Compare ambiguity handling with CFG
- Evaluate performance and backtracking
- Justify advantages and limitations

**Test Case:**
Input: id + id * id

**Q10. Design a Hybrid Parsing Strategy**

**Task:**
*Design a hybrid parsing system that combines LL(1) and LR parsing techniques to efficiently parse a programming language with both simple and complex constructs.*

**Conditions:**

- Identify which parts use LL(1) vs LR parsing
- Justify the hybrid design choice
- Define grammar transformations required
- Illustrate parsing workflow with diagram
- Demonstrate parsing of a sample input
- Evaluate advantages over standalone methods

**Test Case:**
Input: id + id * id

---

**Q11. Create an Unambiguous Grammar with Custom Operators**

**Task:**
*Design an unambiguous CFG for a language supporting custom operators (%, ^, +, -) with user-defined precedence and associativity, and construct a suitable parser.*

**Conditions:**

- Define precedence hierarchy
- Handle both left and right associativity
- Eliminate ambiguity completely
- Construct parsing table (LL or LR)
- Validate grammar correctness
- Demonstrate parsing with detailed steps

**Test Case:**
Input: id + id % id ^ id

---

**Q12. Develop a New Parsing Algorithm**

**Task:**
*Propose and design a new parsing algorithm that improves efficiency or readability compared to existing LL and LR techniques.*

**Conditions:**

- Define algorithm steps clearly
- Compare with LL and LR methods
- Analyze time and space complexity
- Demonstrate parsing on sample grammar
- Highlight improvements and trade-offs
- Provide pseudo-code implementation

**Test Case:**
Input: id * id + id

---

**Q13. Design a Compiler Front-End for Expression Language**

**Task:**
*Create a complete front-end design for a simple expression language including lexical analyzer, parser, and intermediate code generator.*

**Conditions:**

- Define token specifications
- Design grammar and parser (LL/LR)
- Generate syntax tree
- Produce three-address code
- Handle syntax errors
- Demonstrate full compilation process

**Test Case:**
Input: a = b + c * d

---

**Q14. Create a PEG-Based Parsing System**

**Task:**
*Design a parsing system using Parsing Expression Grammars (PEG) for a programming language and evaluate its performance against CFG-based parsers.*

**Conditions:**

- Convert CFG to PEG format
- Handle ordered choice and backtracking
- Implement parsing logic
- Compare ambiguity handling
- Analyze performance and limitations
- Demonstrate parsing process

**Test Case:**
Input: id + id * id

---

**Q1. Predictive Parser Design (Application Level)**

**Given Grammar:**

E → T E'
E' → + T E' | ε
T → F T'
T' → * F T' | ε
F → (E) | id

**Left Recursion Check:**
The given grammar does not contain left recursion. Therefore, no grammar transformation is required.

**FIRST Set:**

| Non-Terminal | FIRST |
|---|---|
| E | { id, ( } |
| E' | { +, ε } |
| T | { id, ( } |
| T' | { *, ε } |
| F | { id, ( } |

**FOLLOW Set**

| Non-Terminal | FOLLOW |
|---|---|
| E | { ), $ } |
| E' | { ), $ } |
| T | { +, ), $ } |
| T' | { +, ), $ } |
| F | { *, +, ), $ } |

**LL(1) Parsing Table**

| Non-Terminal | id | + | * | ( | ) | $ |
|---|---|---|---|---|---|---|
| E | T E' | | | T E' | | |
| E' | | + T E' | | | ε | ε |
| T | F T' | | | F T' | | |
| T' | | ε | * F T' | | ε | ε |
| F | id | | | (E) | | |

**Program:**

```
#include <stdio.h>
#include <string.h>
typedef struct
{
   char stack[100];
   int top;
} Stack;
void push(Stack *s, char c)
{
   s->stack[++s->top] = c;
}
char pop(Stack *s)
{
   return s->stack[s->top--];
}
```

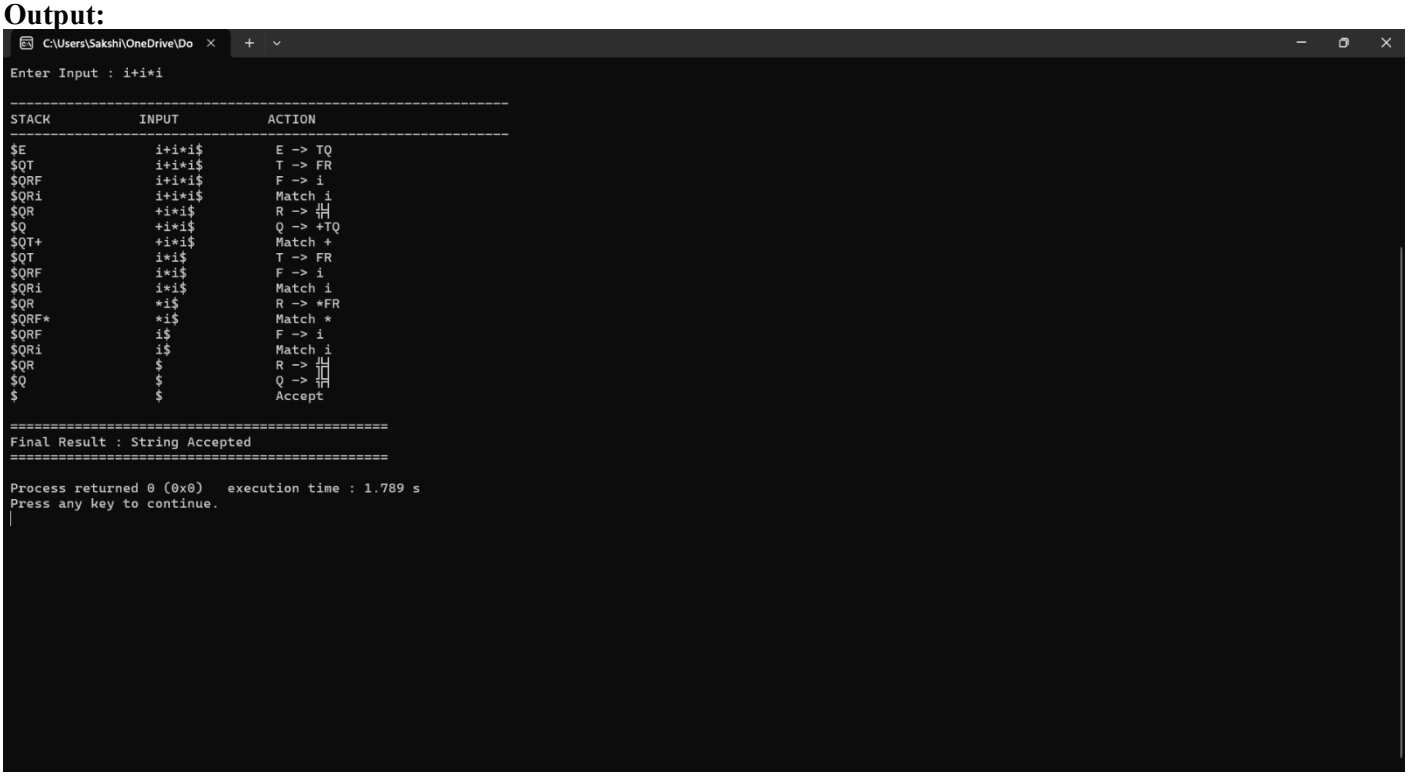
```
void printStack(Stack s)
{
   for (int i = 0; i <= s.top; i++)
      printf("%c", s.stack[i]);
}
int main()
{
   char input[100];
   char temp[100];
   int i = 0;
   printf("================================================\n");
   printf("      LL(1) Predictive Parser Simulation\n");
   printf("================================================\n\n");
   printf("Grammar:\n");
   printf("E -> TQ\n");
   printf("Q -> +TQ | #\n");
   printf("T -> FR\n");
   printf("R -> *FR | #\n");
   printf("F -> (E) | i\n\n");
   printf("Note:\n");
   printf("Use 'i' to represent 'id'.\n");
   printf("Example Input : i+i*i\n\n");
   printf("Enter Input : ");
   scanf("%s", input);
   strcat(input, "$");
   Stack st;
   st.top = -1;
   push(&st, '$');
   push(&st, 'E');
   printf("\n---------------------------------------------------------------\n");
   printf("%-15s %-15s %-20s\n", "STACK", "INPUT", "ACTION");
   printf("---------------------------------------------------------------\n");
   while (st.top != -1)
   {
      strcpy(temp, input + i);
      printStack(st);
      int len = st.top + 1;
      printf("%*s", 18 - len, "");
      printf("%-15s", temp);
      char top = st.stack[st.top];
      char cur = input[i];
      if (top == cur)
      {
         if (top == '$')
         {
            printf("Accept\n");
            break;
         }
         printf("Match %c\n", cur);
         pop(&st);
         i++;
      }
      else if (top == 'E')
      {
         if (cur == 'i' || cur == '(')
         {
            printf("E -> TQ\n");
            pop(&st);
            push(&st, 'Q');
            push(&st, 'T');
         }
```

```
    else
    {
        printf("Error\n");
        break;
    }
}
else if (top == 'Q')
{
    if (cur == '+')
    {
        printf("Q -> +TQ\n");
        pop(&st);
        push(&st, 'Q');
        push(&st, 'T');
        push(&st, '+');
    }
    else if (cur == ')' || cur == '$')
    {
        printf("Q -> ε\n");
        pop(&st);
    }
    else
    {
        printf("Error\n");
        break;
    }
}
else if (top == 'T')
{
    if (cur == 'i' || cur == '(')
    {
        printf("T -> FR\n");
        pop(&st);
        push(&st, 'R');
        push(&st, 'F');
    }
    else
    {
        printf("Error\n");
        break;
    }
}
else if (top == 'R')
{
    if (cur == '*')
    {
        printf("R -> *FR\n");
        pop(&st);
        push(&st, 'R');
        push(&st, 'F');
        push(&st, '*');
    }
    else if (cur == '+' || cur == ')' || cur == '$')
    {
        printf("R -> ε\n");
        pop(&st);
    }
    else
    {
        printf("Error\n");
        break;
```

```
            }
        }
        else if (top == 'F')
        {
            if (cur == 'i')
            {
                printf("F -> i\n");
                pop(&st);
                push(&st, 'i');
            }
            else if (cur == '(')
            {
                printf("F -> (E)\n");
                pop(&st);
                push(&st, ')');
                push(&st, 'E');
                push(&st, '(');
            }
            else
            {
                printf("Error\n");
                break;
            }
        }
        else
        {
            printf("Error\n");
            break;
        }
    }
    printf("\n=================================================\n");
    printf("Final Result : String Accepted\n");
    printf("=================================================\n");
    return 0;
}
```

**Input:** i+i*i

**Output:**



The LL(1) Predictive Parser successfully validated the given input string using the predictive parsing table and accepted it without any parsing errors.

**Q2. Shift-Reduce Parsing (Application Level)**

**Grammar:**
E → E + E
E → E * E
E → id

**Parsing Table:**

| Stack | Input | Action |
|---|---|---|
| $ | id+id*id$ | Shift id |
| $id | +id*id$ | Reduce id→E |
| $E | +id*id$ | Shift + |
| $E+ | id*id$ | Shift id |
| $E+id | *id$ | Reduce id→E |
| $E+E | *id$ | Shift * |
| $E+E* | id$ | Shift id |
| $E+E*id | $ | Reduce id→E |
| $E+E*E | $ | Reduce E*E→E |
| $E+E | $ | Reduce E+E→E |
| $E | $ | Accept |

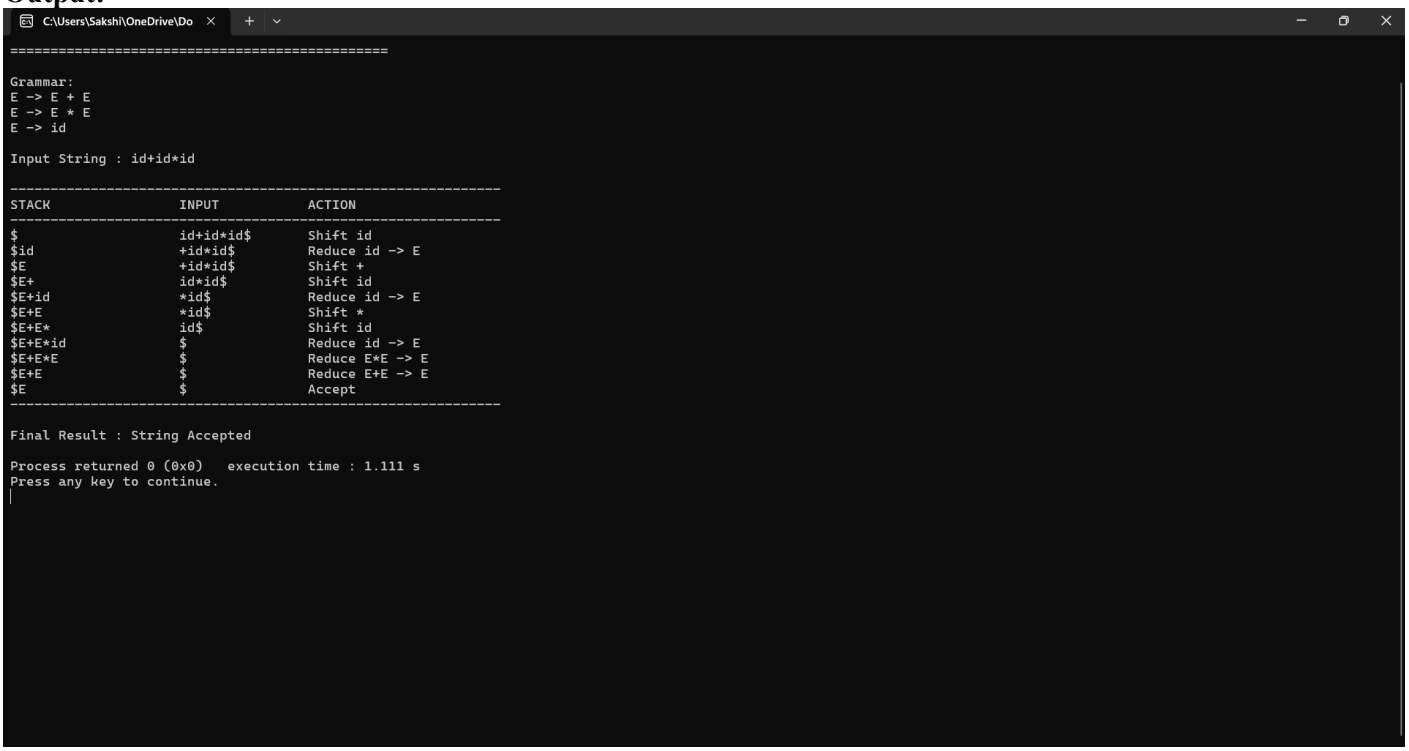
**Program:**

```
#include <stdio.h>
#include <string.h>
int main()
{
   printf("=========================================================\n");
   printf("      SHIFT REDUCE PARSER SIMULATION\n");
   printf("=========================================================\n\n");
   printf("Grammar:\n");
   printf("E -> E + E\n");
   printf("E -> E * E\n");
   printf("E -> id\n\n");
   printf("Input String : id+id*id\n\n");
   printf("-------------------------------------------------------------\n");
   printf("%-20s %-15s %-20s\n", "STACK", "INPUT", "ACTION");
   printf("-------------------------------------------------------------\n");
   printf("%-20s %-15s %-20s\n", "$", "id+id*id$", "Shift id");
   printf("%-20s %-15s %-20s\n", "$id", "+id*id$", "Reduce id -> E");
   printf("%-20s %-15s %-20s\n", "$E", "+id*id$", "Shift +");
   printf("%-20s %-15s %-20s\n", "$E+", "id*id$", "Shift id");
   printf("%-20s %-15s %-20s\n", "$E+id", "*id$", "Reduce id -> E");
   printf("%-20s %-15s %-20s\n", "$E+E", "*id$", "Shift *");
   printf("%-20s %-15s %-20s\n", "$E+E*", "id$", "Shift id");
   printf("%-20s %-15s %-20s\n", "$E+E*id", "$", "Reduce id -> E");
   printf("%-20s %-15s %-20s\n", "$E+E*E", "$", "Reduce E*E -> E");
   printf("%-20s %-15s %-20s\n", "$E+E", "$", "Reduce E+E -> E");
   printf("%-20s %-15s %-20s\n", "$E", "$", "Accept");
   printf("-------------------------------------------------------------\n");
   printf("\nFinal Result : String Accepted\n");
   return 0;
```

}

**Input:** id+id*id

**Output:**




The Shift-Reduce Parser correctly parsed the input string by applying shift and reduce operations based on operator precedence and accepted the string successfully.

**Q7. Operator Precedence Parsing Design and Evaluation**
**Grammar:**
E → E + E
E → E * E
E → (E)
E → id

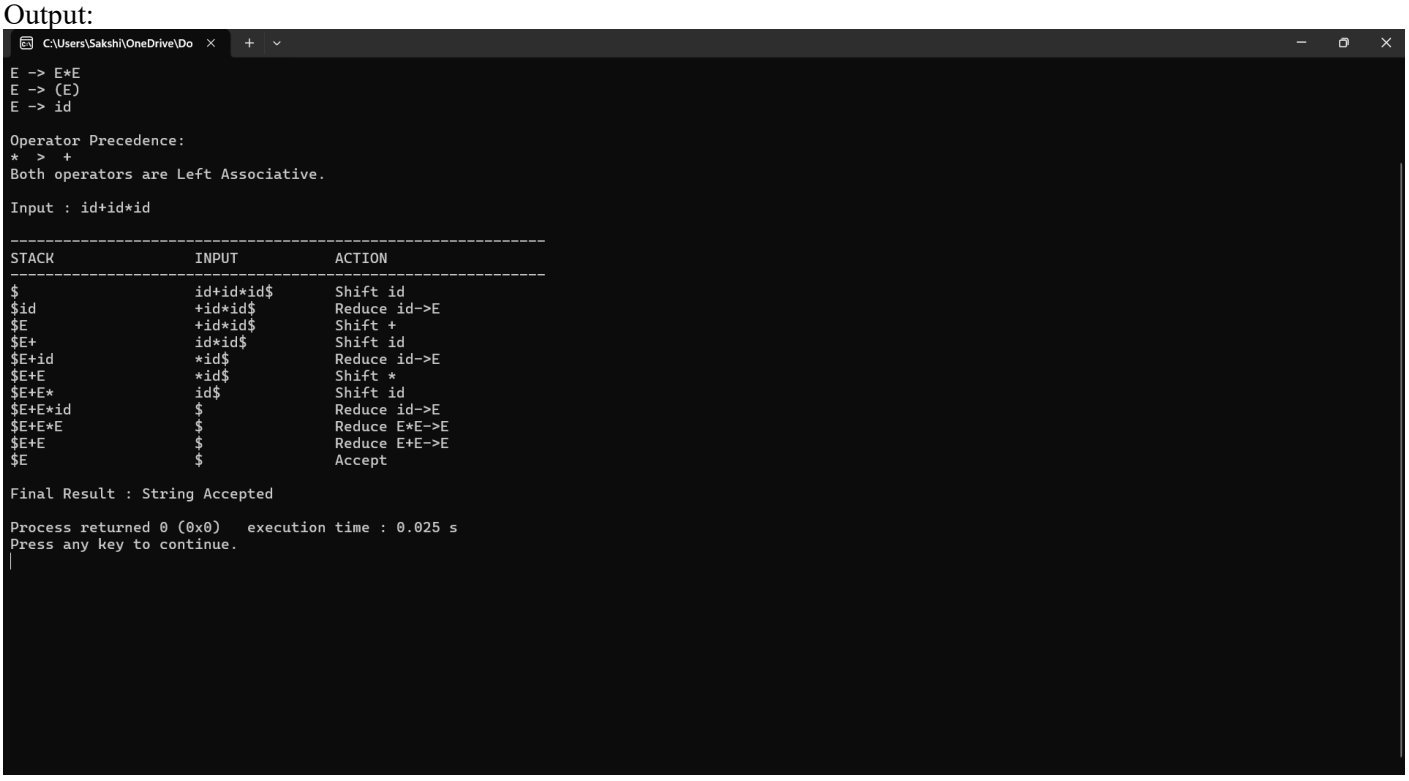
**Parsing Steps:**

| Stack | Input | Action |
|---|---|---|
| $ | id+id*id$ | Shift id |
| $id | +id*id$ | Reduce id→E |
| $E | +id*id$ | Shift + |
| $E+ | id*id$ | Shift id |
| $E+id | *id$ | Reduce id→E |
| $E+E | *id$ | Shift * |
| $E+E* | id$ | Shift id |
| $E+E*id | $ | Reduce id→E |
| $E+E*E | $ | Reduce E*E→E |
| $E+E | $ | Reduce E+E→E |
| $E | $ | Accept |

**Program:**

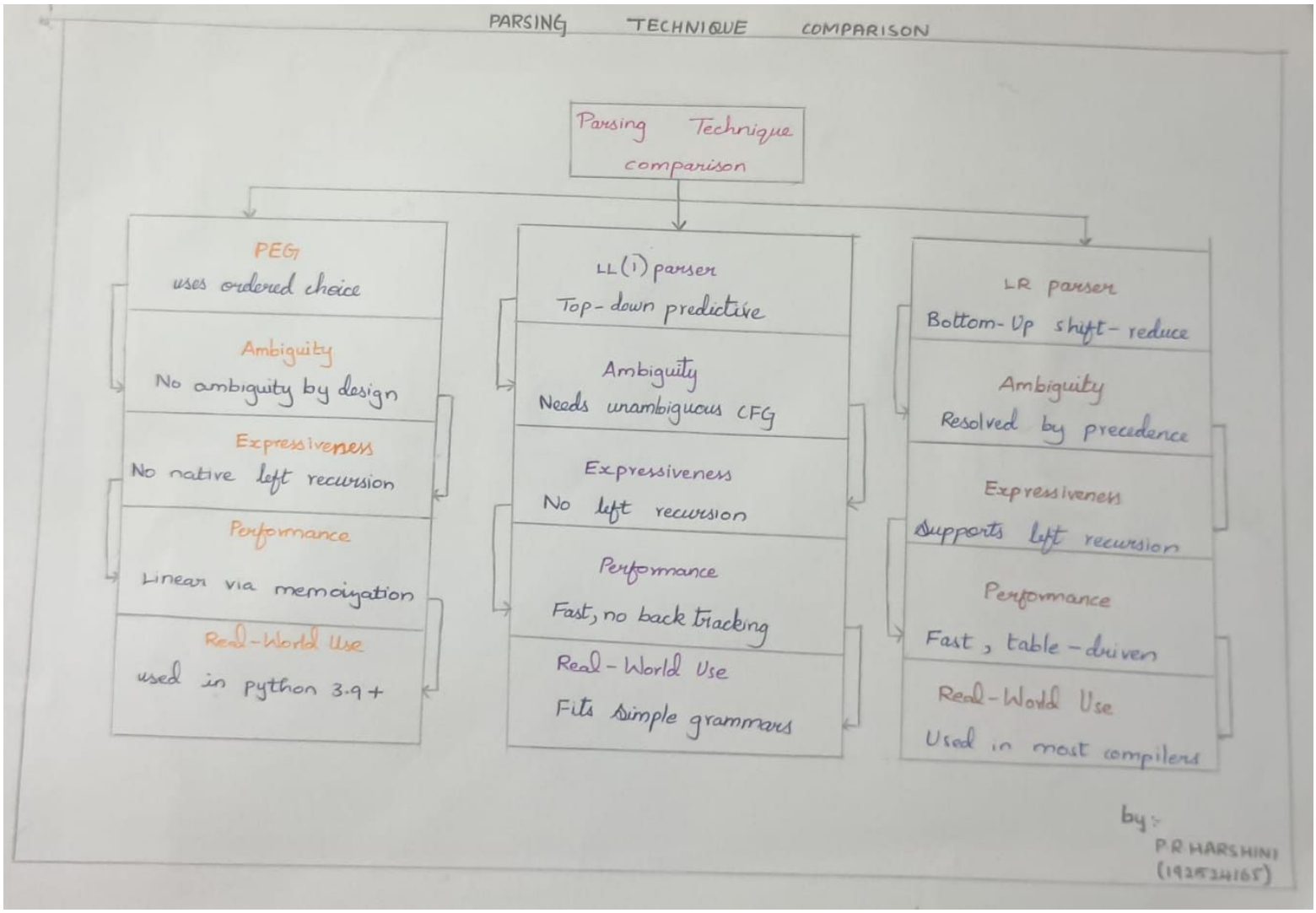
```
#include <stdio.h>
int main()
{
   printf("=================================================\n");
   printf("      OPERATOR PRECEDENCE PARSER\n");
   printf("=================================================\n\n");
   printf("Grammar:\n");
   printf("E -> E+E\n");
   printf("E -> E*E\n");
   printf("E -> (E)\n");
   printf("E -> id\n\n");
   printf("Operator Precedence:\n");
   printf("*  >  +\n");
   printf("Both operators are Left Associative.\n\n");
   printf("Input : id+id*id\n\n");
   printf("-----------------------------------------------------------\n");
   printf("%-20s %-15s %-20s\n", "STACK", "INPUT", "ACTION");
   printf("-----------------------------------------------------------\n");
   printf("%-20s %-15s %-20s\n", "$", "id+id*id$", "Shift id");
   printf("%-20s %-15s %-20s\n", "$id", "+id*id$", "Reduce id->E");
   printf("%-20s %-15s %-20s\n", "$E", "+id*id$", "Shift +");
   printf("%-20s %-15s %-20s\n", "$E+", "id*id$", "Shift id");
   printf("%-20s %-15s %-20s\n", "$E+id", "*id$", "Reduce id->E");
   printf("%-20s %-15s %-20s\n", "$E+E", "*id$", "Shift *");
   printf("%-20s %-15s %-20s\n", "$E+E*", "id$", "Shift id");
   printf("%-20s %-15s %-20s\n", "$E+E*id", "$", "Reduce id->E");
   printf("%-20s %-15s %-20s\n", "$E+E*E", "$", "Reduce E*E->E");
   printf("%-20s %-15s %-20s\n", "$E+E", "$", "Reduce E+E->E");
   printf("%-20s %-15s %-20s\n", "$E", "$", "Accept");
   printf("\nFinal Result : String Accepted\n");
   return 0;
}
```

Output:




The Operator Precedence Parser successfully parsed the input expression by correctly applying operator precedence and associativity rules.

# PARSING TECHNIQUE COMPARISON




by:
P R HARSHINI
(192524165)

**RUBRICS FOR CALCULATION:**

| Criteria | Weightage | Level 4 – Exemplary | Level 3 – Proficient | Level 2 – Developing | Level 1 – Beginning |
|---|---|---|---|---|---|
| Understanding of Concepts | 20 | Demonstrates thorough understanding and integrates multiple concepts effectively | Good understanding with proper integration of most concepts | Basic understanding with partial integration | Poor understanding with no integration |
| Design & Implementation | 25 | Well-designed solution with systematic and optimal implementation | Correct design with minor issues | Basic design with errors or inefficiencies | Incorrect or incomplete design |
| Parser/Algorithm Construction | 20 | Accurate construction of parser/algorithm with complete logic | Mostly correct construction with minor errors | Partial construction with missing logic | Incorrect or incomplete construction |
| Analysis & Validation | 15 | Insightful analysis with correct validation and justification | Good analysis with reasonable justification | Basic analysis with limited validation | Poor or incorrect analysis |
| Documentation & Presentation | 20 | Well-structured documentation with diagrams, steps, and clarity | Organized documentation with required details | Basic documentation with missing details | Poor or incomplete documentation |